AI-Assisted Hair Care Recommendation System: A Hybrid Decision Support System Using Large Language Models

Quarshona Collins
Department of Computer Science
North Carolina A&T State University
Greensboro, NC, USA
qcollins@aggies.ncat.edu

Courtney Carr
Department of Computer Science
North Carolina A&T State University
Greensboro, NC, USA
ccarr2@aggies.ncat.edu

***Abstract*—This paper covers the design and development of an AI-Assisted Hair Care Recommendation System. The system combines a rule-based processing module we call the "Mini Brain" with a Large Language Model (LLM) to give users personalized hair care advice. When someone fills out our survey, the system reads their answers, figures out what hair issues they have, and sends all of that to GPT-4o-mini. The AI then comes back with a full routine, product suggestions, ingredients to look for, things to avoid, and an explanation of why it made those choices. We built the interface using Streamlit and connected it to the OpenAI API. This paper is split into two phases: Phase 1 covers the problem we identified, our research, and our first working prototype. Phase 2 goes deeper into the system design, adds UML diagrams, and shows our improved version of the app.**

***Index Terms*—Recommender Systems, Large Language Models, Natural Language Processing, Personalized Hair Care, Decision Support Systems, Explainable AI, Hybrid Systems**

— PHASE 1: REQUIREMENTS, LITERATURE & INITIAL PROTOTYPE —

I. Introduction

Walk into any beauty store and you will find hundreds of hair care products on the shelf, all promising different things. For most people, picking the right ones is basically a guessing game. You try something, it does not work, you waste money, and sometimes your hair ends up worse than before. Getting real advice from a dermatologist or hair specialist is expensive and not realistic for most people.

That is the problem we wanted to solve. We built an AI system that asks users a few simple questions about their hair and gives back a personalized routine and product recommendations, along with an explanation of why those things were suggested. The whole idea is to make professional-level hair advice accessible to anyone with a phone or laptop.

Our system pulls from research in recommendation systems [1], [2], AI personalization [3], modern language models [4], and explainable AI [5] to make sure the advice it gives is actually grounded in real methods and not just made up.

II. Problem Statement

A lot of people struggle to find the right hair products because their hair is complicated. Someone might have curly, low-porosity hair that is also color-treated and dry — and finding products that work for all of those things at once is really hard. Searching online usually just gives you generic tips that do not account for your specific situation.

Some of the biggest problems we saw:

- People waste a lot of money buying products that end up not working
- Using the wrong products can actually damage your hair over time
- There is no easy starting point for someone who just wants a simple routine
- A lot of the advice online is either too vague or contradicts itself

We wanted to build something that actually listens to what you tell it about your hair and gives you a real, personalized answer instead of the same generic advice everyone else gets.

III. Research Objectives

Here is what we set out to do with this project:

1) Build a questionnaire that collects the right information about someone's hair
2) Use an AI model to understand and interpret what the user tells us
3) Figure out what hair conditions or issues the user has
4) Recommend specific product types and ingredients based on those conditions
5) Give the user a step-by-step routine they can actually follow
6) Make sure the system explains *why* it made each recommendation

IV. Justification for LLM-Based Approach

We could have built this with a giant list of if-then rules — like, “if hair is coily AND dry AND chemically treated, then recommend X.” But that gets complicated really fast. Hair care involves so many combinations of conditions that you would need thousands of rules just to cover the most common situations, and even then you would miss a lot.

Large Language Models like GPT-4o-mini are much better at handling that kind of complexity. They can read a full description of someone’s hair situation and reason through it the same way a knowledgeable person would. Instead of just matching to a rule, the AI actually thinks about what would make sense for that person.

We still kept a rule-based piece in the system — our “Mini Brain” module — to handle basic input checking and flag common conditions before the AI even sees the input. That way, the AI is not starting from scratch every time and the output is more consistent. This kind of hybrid approach is backed up by research in clinical AI systems [3] where combining rules with AI gives better results than using either one alone.

V. Preliminary Literature Review

A. Recommendation Systems

Roy and Dutta (2022) looked at how recommendation systems work across different fields and found that the best ones use detailed information about the user, not just generic preferences [1]. That is exactly why we built a questionnaire instead of just letting users type in a free-form question — we wanted structured, useful input.

B. Deep Learning-Based Recommenders

Zhang et al. (2019) showed that AI models can take in multiple features about a user at once and combine them to make better recommendations [2]. In our case, those features are things like hair type, scalp condition, and whether the user uses heat tools. The LLM handles all of those together instead of looking at them one by one.

C. NLP Personalization in Clinical Decision Support

Alharbi et al. looked at how NLP can be used in healthcare to give people more personalized advice [3]. Even though they were working in a medical context, the same idea applies here — the more the system understands about the person, the better the recommendation it can give.

D. BERT and Transformer Foundation Models

Devlin et al. (2019) introduced BERT, which was one of the first models to really show how powerful transformer-based AI could be at understanding language [4]. GPT-4o-mini, which powers our system, is built on the same kind of foundation and is what allows it to understand complex hair descriptions and respond intelligently.

E. Explainable AI for Decision Support

Holzinger et al. (2019) made the point that AI systems people actually trust need to be able to explain themselves [5]. We took that seriously — our system does not just give you a list of products, it tells you exactly why it is recommending them based on what you said about your hair.

VI. Methodology

We followed an Agile approach, working in short sprints so we could keep testing and improving as we went. Here is how we broke it down:

- **Sprint 1:** Designed the questionnaire and started mapping out the system with use case and DFD diagrams
- **Sprint 2:** Wrote and tested different versions of the AI prompt to see which gave the best results
- **Sprint 3:** Built the Streamlit interface and connected everything to the OpenAI API
- **Sprint 4 (13-day testing phase):** Tested the full system and evaluated how well it personalized, explained, and presented recommendations

The basic flow of how the system works is: the user fills out the survey, the Mini Brain scans their answers for key conditions, those conditions get packaged into a structured prompt, and the AI returns a full recommendation. We did not need any outside datasets since the AI generates everything on the fly based on what the user tells it.

VII. System Design

The system is built in three main parts that each have a specific job: the Streamlit front-end that the user interacts with, the Python backend that processes the input and builds the prompt, and GPT-4o-mini that actually generates the recommendation. Keeping these three parts separate made the system easier to build, test, and update.

A. Functional Decomposition

Fig. 1 shows how the system breaks down into smaller pieces. At the top level, the system does three things: takes in user input, builds a response, and delivers that response to the user. Each of those breaks down further — for example, “Take User Input” includes verifying that the input is valid before anything else happens.

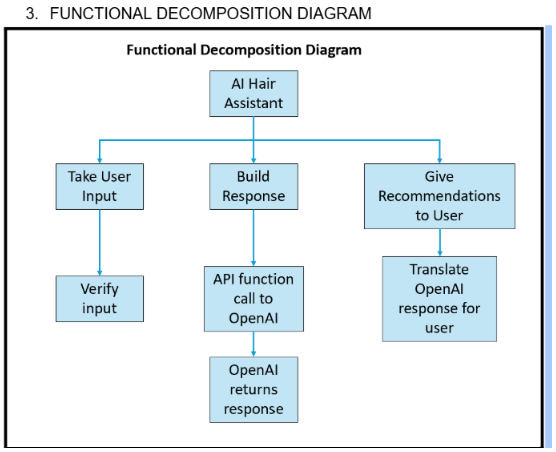

Fig. 1. Functional Decomposition Diagram of the AI Hair Assistant System

B. Data Flow Diagrams

Fig. 2 shows three levels of data flow through the system. The top-level context diagram just shows the user sending input and getting recommendations back. The Level 0 DFD breaks that into the three main processes. The lower-level DFD zooms in on the "Build Response" step and shows how the system constructs the prompt and calls the OpenAI API.

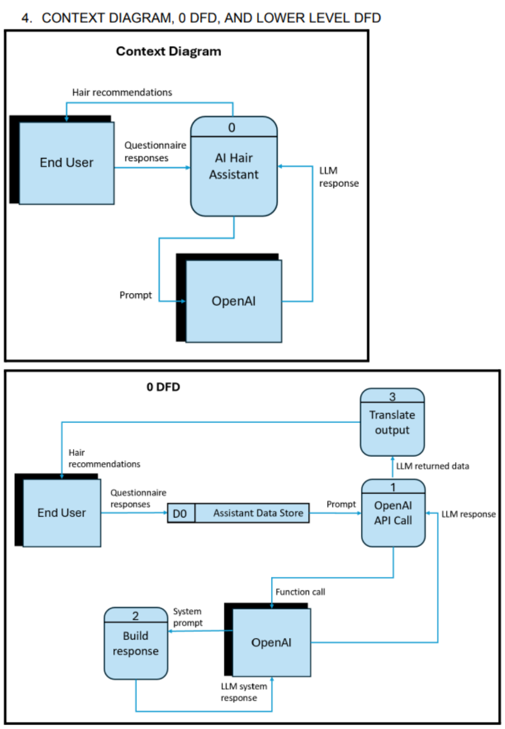

Fig. 2. Context Diagram (top), Level 0 DFD (middle), and Lower-Level DFD (bottom)

C. Decision Table — Input Verification

Before anything gets sent to the AI, the Mini Brain checks whether the input is actually usable. Table I shows the logic behind that check — there are three conditions, and depending on which ones are true, the input is either accepted or rejected.

TABLE I
DECISION TABLE — INPUT VERIFICATION LOGIC

Verify User Input	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Dropdown menus filled out	Y	–	–	Y	–	Y	N	–
Description typed in text box	–	Y	–	Y	–	–	Y	–
Input is about hair condition	N	N	Y	Y	N	Y	Y	N
Accept input			X	X		X	X	
Reject input	X	X			X			X

VIII. LLM INTEGRATION AND PROMPT ENGINEERING

The way we talk to the AI matters a lot. If the prompt is vague, the output will be vague. So we put a lot of thought into how to structure it. Every prompt has four parts: we tell the AI what role it is playing (a hair care advisor), we give it all of the user's information, we include any flags the Mini Brain detected, and we tell it exactly what format to respond in.

A. Fine-Tuning Strategy

We used GPT-4o-mini straight out of the box — no custom training. We looked into fine-tuning but decided against it for a few reasons: there is no publicly available labeled dataset for hair care recommendations, the pre-trained model already performed really well in our tests, and training a custom model would have taken way more time than we had. In the future, we could look at using retrieval-augmented generation (RAG) to pull from a real product database.

B. Prompt Template

Here is the exact prompt structure we used:

```
prompt = f"""
You are a professional hair care advisor.
Hair Type: {hair_type}
Scalp Condition: {scalp_condition}
Hair Goal: {hair_goal}
Heat Styling Frequency: {heat_usage}
Chemical Treatments: {chemical_treatments}
Additional Description: {user_description}
Based on these inputs, provide:
1. Daily and Weekly Routine
2. Best Product Types to Use
3. Key Ingredients to Look For
4. Things to Avoid
5. Explanation: Why I Recommend This
"""
```

IX. PROTOTYPE IMPLEMENTATION — INITIAL VERSION

A. Initial (Original) Prototype

The first version of the app was mostly about proving the concept worked. The user picked answers from dropdown menus, hit a button, and got a recommendation back from the AI. It was pretty bare-bones but it showed us the core idea was solid. The screenshots below are from that original version.

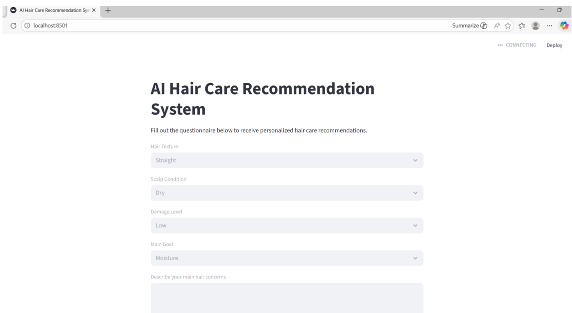

Fig. 3. Initial Prototype — Questionnaire Interface

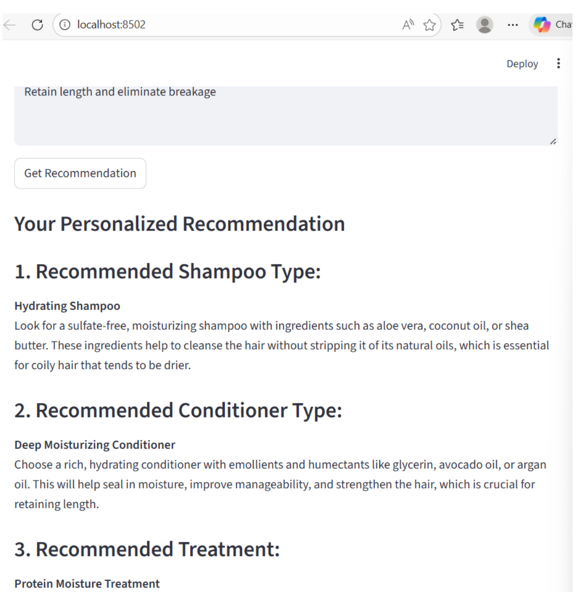

Fig. 4. Initial Prototype — Recommendation Output (Part 1)

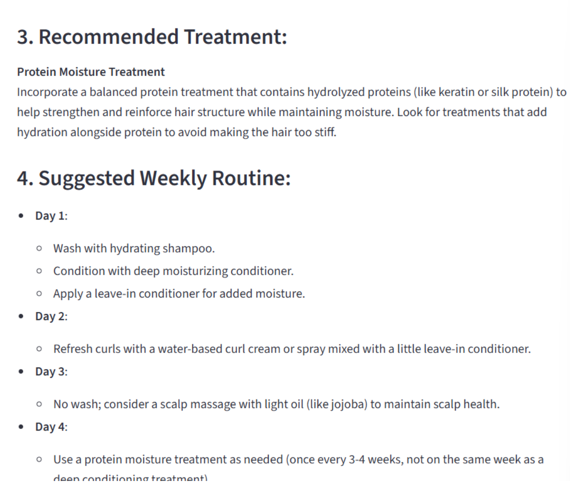

Fig. 5. Initial Prototype — Recommendation Output (Part 2)

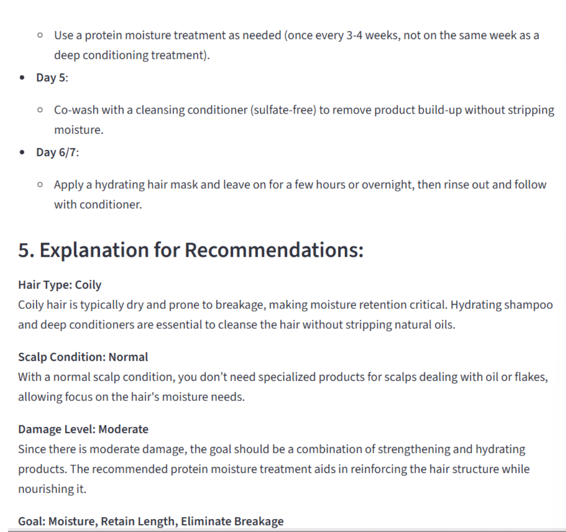

Fig. 6. Initial Prototype — Weekly Routine Section

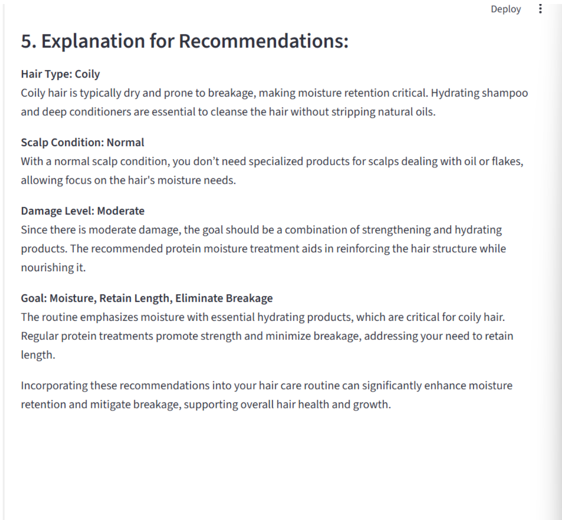

Fig. 7. Initial Prototype — Explanation Section

— PHASE 2: SYSTEM DESIGN, DEVELOPMENT & ENHANCED PROTOTYPE —

X. Full System Design — Phase 2

Phase 2 is where we got into the formal system design work. We created all the UML diagrams, finalized the architecture, and updated the prototype based on what we learned from testing. This section walks through each of the design artifacts we produced.

A. Object Relationship Diagram

The overall system is organized into three layers, each with a specific job. Fig. 8 shows how these layers connect to each other.

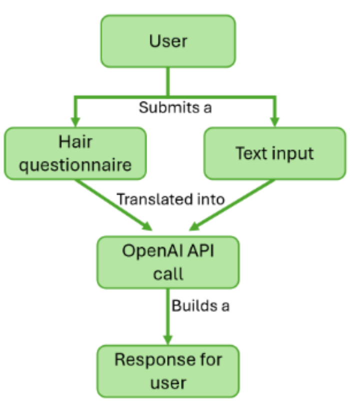

Fig. 8. System Architecture Diagram — Three-Tier Design

TABLE II
System Architecture Layers

Layer	Component	What It Does
Presentation	Streamlit UI	Shows the survey, collects answers, displays the recommendation
Processing	Python Backend + Mini Brain	Validates input, detects hair conditions, builds the AI prompt
Intelligence	GPT-4o-mini (OpenAI)	Reads the prompt and generates the personalized recommendation

B. Use Case Diagram

The Use Case Diagram (Fig. 9) shows everything a user can do with the system and how those actions connect to what happens on the back end.

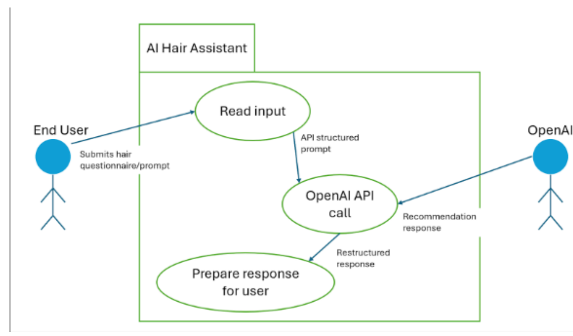

Fig. 9. Use Case Diagram — System Functionalities

C. Class Diagram

The Class Diagram (Fig. 10) shows the main objects in the system, what information they hold, and what they can do.

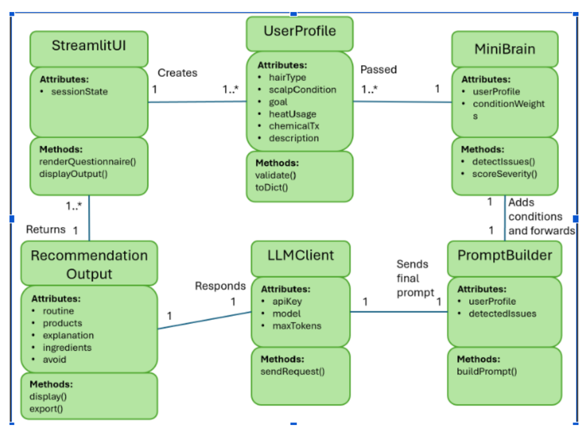

Fig. 10. Class Diagram — Key Entities, Attributes, and Relationships

D. E-R Diagram

The Entity-Relationship Diagram (Fig. 11) shows how the data in the system is organized and how the different pieces relate to each other.

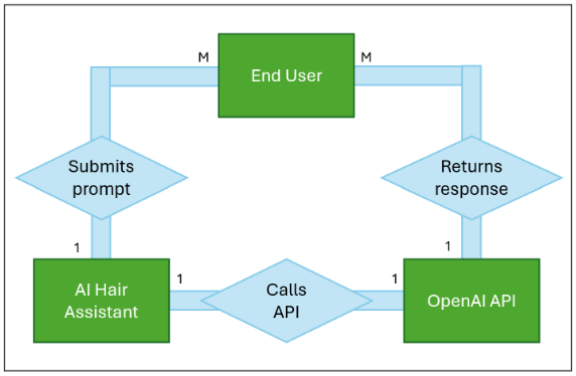

Fig. 11. Entity-Relationship Diagram

E. Extended Decision Table

Table III shows how specific combinations of hair type, scalp condition, and goal map to different recommendation priorities.

TABLE III
EXTENDED DECISION TABLE — INPUT CONDITIONS TO RECOMMENDATION FOCUS

Hair Type	**Scalp**	**Goal**	**Recommendation Focus**
Curly	Dry	Moisture	Deep conditioning, leave-in cream, avoid sulfates
Straight	Oily	Frizz control	Clarifying shampoo, light serum, volumizing routine
Coily	Sensitive	Hair growth	Scalp treatment, protein-free moisturizers, gentle cleanse
Wavy	Normal	Damage repair	Protein treatment, bond repair serum, heat protectant
Any + Chemical Tx	Any	Any	Bond-building products, reduced heat, strengthening masks

XI. Enhanced Prototype — Phase 2

After testing the first version and getting some feedback, we went back and made a lot of improvements. The biggest changes were adding the optional text box so users can describe their hair in their own words, cleaning up the layout, and restructuring the AI output into five clearly labeled sections so it is easier to read and follow.

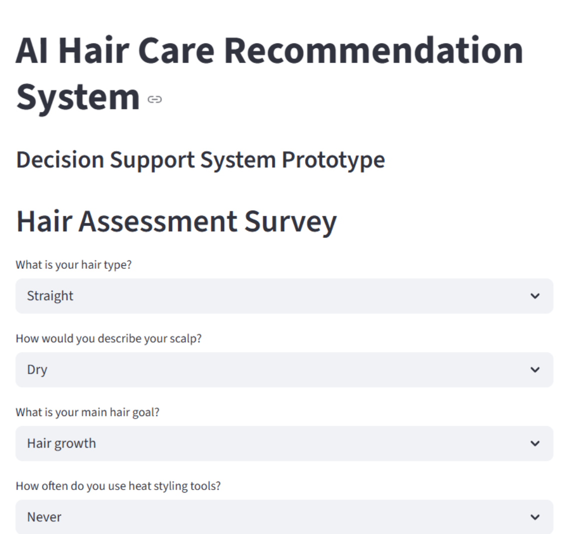

Fig. 12. Updated Survey — Top Section

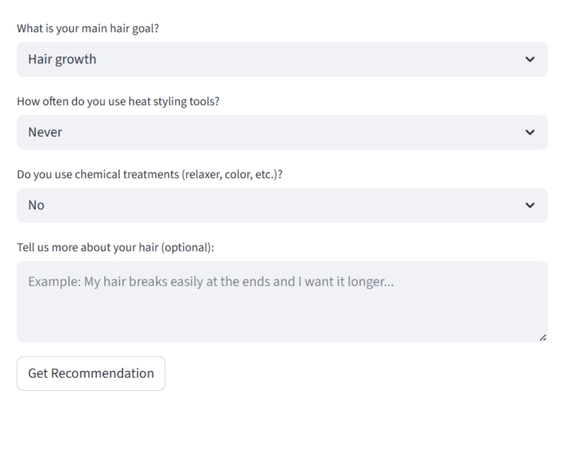

Fig. 13. Updated Survey — Bottom Section with Optional Description Box

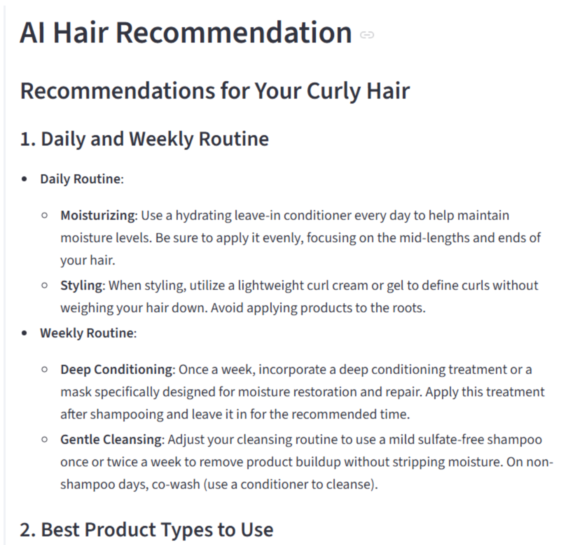

Fig. 14. Updated Output — Section 1: Daily and Weekly Routine

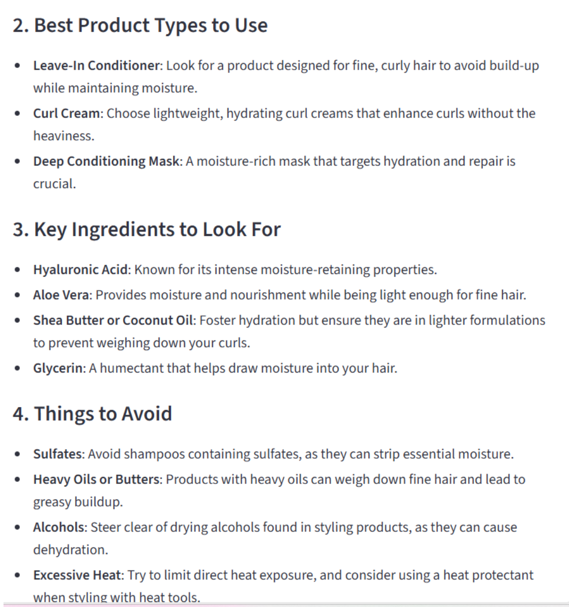

Fig. 15. Updated Output — Sections 2–4: Products, Ingredients, Things to Avoid

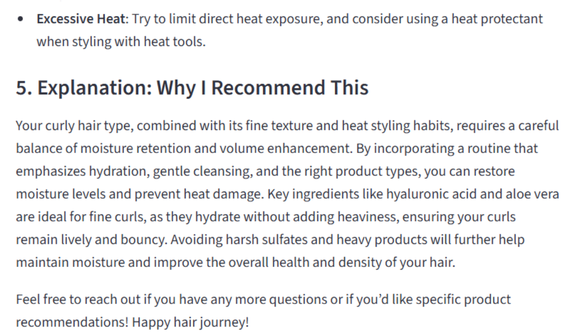

Fig. 16. Updated Output — Section 5: Explanation

Here is a summary of what changed between the two versions:

- Added an optional free-text box so users can give extra context in their own words
- Added the Mini Brain module to pre-process inputs before they reach the AI
- Restructured the AI output into five numbered sections that are much easier to read
- Cleaned up the overall layout so the app looks more polished
- Improved the prompt so the AI responses are more detailed and better organized

— PHASE 3: FINAL IMPLEMENTATION, EVALUATION & FUTURE WORK —

XII. Final System Enhancements — Phase 3

Phase 3 focused on expanding the system into a fully featured, production-quality prototype. The major upgrades include a significantly longer 13-question survey, an enhanced dual Mini Brain architecture with weighted scoring, a visual Hair Profile Score Dashboard, structured AI output with day-by-day routines, confidence scoring, and a user feedback loop.

The final questionnaire now collects detailed information on hair porosity, density, length, wash frequency, damage level, environment, styling habits, diet & water intake, and an optional free-text description. This richer input allows for much more accurate personalization.

The Mini Brain was upgraded with both a simple rule-based detector and an enhanced weighted scoring engine that calculates four key metrics (Moisture Level, Damage Level, Scalp Health, and Breakage Risk) displayed as a clear dashboard with progress bars. Combined condition flags were also added for high-risk situations.

After preprocessing, a carefully structured prompt is sent to GPT-4o-mini, which returns a clean recommendation organized into five clearly labeled sections. The interface includes visual confidence scoring and a 1–5 star feedback loop.

The screenshots below show the final v4.0 system:

Fig. 17. Final v4.0 Hair Assessment Survey (Top Section)

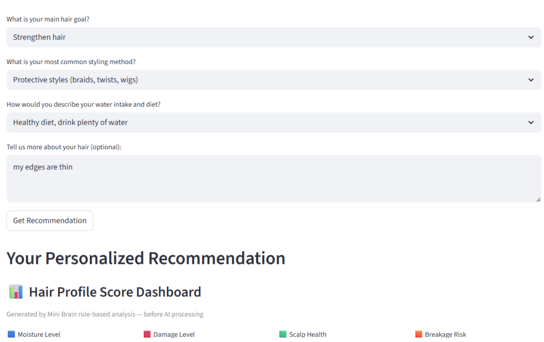

Fig. 18. Final v4.0 Hair Assessment Survey (Bottom Section)

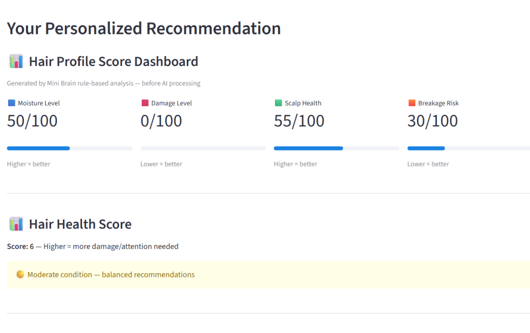

Fig. 19. Final Hair Profile Score Dashboard

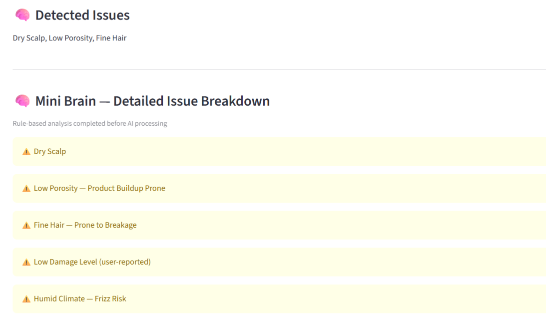

Fig. 20. Final Recommendation - Suggested Weekly Routine

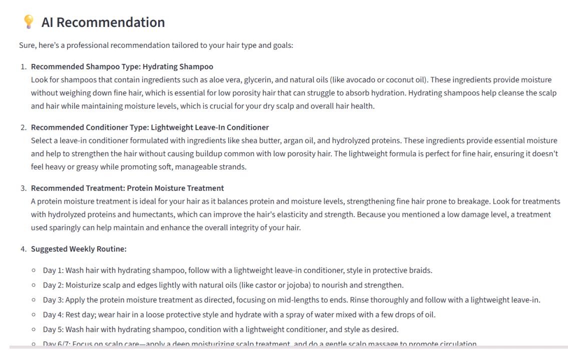

Fig. 21. Final Recommendation - Products and Ingredients

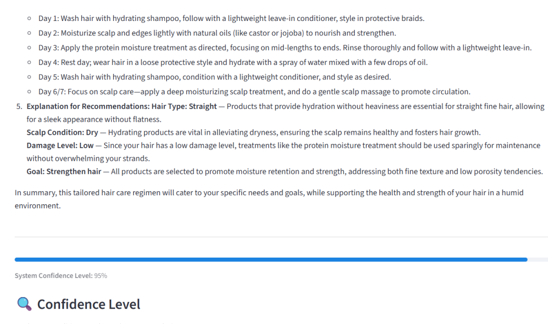

Fig. 22. Final Recommendation - Explanation Section

The complete source code for this final version is available in the project repository (file: `app.py`) and is also reproduced in full in Section XIII.

XIII. Code Documentation

The final application is contained in a single `app.py` file and includes the expanded questionnaire, dual Mini Brain functions, Hair Profile Score Dashboard, structured output parsing, and sidebar documentation of the full system process. The complete source code is provided below.

Demo Video: A recorded demonstration of the final v4.0 system is available at: https://drive.google.com/file/d/1Fqz2u9xVQ9YAS1ODEFZOdeCQiZ22qpVr/view

```python
import streamlit as st
import openai

# ======================
# PAGE CONFIG
# ======================
st.set_page_config(
    page_title="AI Hair Care System",
    page_icon="",
    layout="wide"
)

st.title(" AI Hair Care Recommendation System")
st.subheader("Decision Support System Prototype - v4.0")

# ======================
# API KEY
# ======================
api_key = "YOUR_API_KEY_HERE"

# ======================
# USER INPUT SECTION
# ======================
st.header("Hair Assessment Survey")

col1, col2 = st.columns(2)

with col1:
    hair_type = st.selectbox(
        "What is your hair type?",
        ["Straight", "Wavy", "Curly", "Coily"]
    )
    scalp_condition = st.selectbox(
        "How would you describe your scalp?",
        ["Dry", "Oily", "Normal", "Sensitive"]
    )
    porosity = st.selectbox(
        "What is your hair porosity?",
        ["Low", "Medium", "High"]
    )
    hair_length = st.selectbox(
        "What is your current hair length?",
        ["Short (ear length or above)",
         "Medium (shoulder length)",
         "Long (past shoulder)",
         "Very Long (past chest)"]
    )
    wash_frequency = st.selectbox(
        "How often do you wash your hair?",
        ["Daily", "Every 2-3 days",
         "Once a week", "Every 2 weeks or less"]
    )

with col2:
    density = st.selectbox(
        "What is your hair density?",
        ["Fine", "Medium", "Thick"]
    )
    heat_usage = st.selectbox(
        "How often do you use heat styling tools?",
        ["Never", "Sometimes", "Often"]
    )
    chemical_treatments = st.selectbox(
        "Do you use chemical treatments?",
        ["No", "Occasionally", "Frequently"]
    )
    damage_level = st.selectbox(
        "How would you rate your current damage level?",
        ["None", "Low", "Moderate", "High"]
    )
    environment = st.selectbox(
        "What type of environment do you live in?",
        ["Humid", "Dry", "Cold", "Mixed/Moderate"]
    )

hair_goal = st.selectbox(
    "What is your main hair goal?",
    ["Hair growth", "Moisture", "Damage repair",
     "Frizz control", "Scalp health",
     "Length retention", "Strengthen hair"]
)

styling_habits = st.selectbox(
    "What is your most common styling method?",
    ["Protective styles (braids, twists, wigs)",
     "Heat styling (flat iron, blow dryer)",
     "Air dry / wash and go",
     "Roller sets / flexi rods",
     "No particular style"]
)

diet_water = st.selectbox(
    "How would you describe your water intake and diet?",
    ["Healthy diet, drink plenty of water",
     "Average diet, moderate water",
     "Poor diet, low water intake",
     "Prefer not to say"]
)

user_description = st.text_area(
    "Tell us more about your hair (optional):",
    placeholder="Example: My hair breaks easily at the ends ..."
)

# ======================
# MINI BRAIN v2.0
# ======================
def detect_hair_issues(scalp, heat, chemical, porosity, density):
    issues = []
    confidence = 70
    if scalp == "Dry":
        issues.append("Dry Scalp")
        confidence += 10
    if scalp == "Oily":
        issues.append("Oily Scalp")
        confidence += 10
    if heat == "Often":
        issues.append("Heat Damage")
        confidence += 15
    if chemical == "Frequently":
        issues.append("Chemical Damage")
        confidence += 15
    if porosity == "Low":
        issues.append("Low Porosity")
        confidence += 10
    if density == "Fine":
        issues.append("Fine Hair")
        confidence += 8
    confidence = min(confidence, 95)
    issues_str = ", ".join(issues) if issues else "No major issues detected"
    return issues_str, confidence

# ======================
# MINI BRAIN v3/v4 Enhanced
# ======================
def run_mini_brain(scalp, heat, chemical, porosity,
                   density, hair_type, goal, damage, environment):
    issues = []
    confidence = 70
    simple_score = 0

    if scalp == "Dry":
        issues.append("Dry Scalp")
        confidence += 10
        simple_score += 2
    elif scalp == "Oily":
        issues.append("Oily Scalp / Product Buildup Risk")
        confidence += 10
        simple_score += 1
    elif scalp == "Sensitive":
        issues.append("Scalp Sensitivity")
        confidence += 6
        simple_score += 1

    if heat == "Often":
        issues.append("Heat Damage Risk")
        confidence += 15
        simple_score += 3
    elif heat == "Sometimes":
        issues.append("Moderate Heat Exposure")
        confidence += 5
        simple_score += 1
```

```
    if chemical == "Frequently":
        issues.append("Chemical Damage")
        confidence += 15
        simple_score += 3
    elif chemical == "Occasionally":
        issues.append("Mild Chemical Processing")
        confidence += 8
        simple_score += 1

    if porosity == "Low":
        issues.append("Low Porosity - Product Buildup Prone")
        confidence += 10
        simple_score += 1
    elif porosity == "High":
        issues.append("High Porosity - Moisture Loss Risk")
        confidence += 10
        simple_score += 2

    if density == "Fine":
        issues.append("Fine Hair - Prone to Breakage")
        confidence += 8
        simple_score += 1

    if damage == "High":
        issues.append("High Damage Level (user-reported)")
        confidence += 10
        simple_score += 3
    elif damage == "Moderate":
        issues.append("Moderate Damage Level (user-reported)")
        confidence += 5
        simple_score += 2
    elif damage == "Low":
        issues.append("Low Damage Level (user-reported)")
        simple_score += 1

    if environment == "Dry":
        issues.append("Dry Climate - Increased Moisture Loss")
        simple_score += 1
    elif environment == "Humid":
        issues.append("Humid Climate - Frizz Risk")
        simple_score += 1

    if goal == "Damage repair":
        issues.append("Existing Damage (user-reported goal)")
        simple_score += 2
    elif goal == "Length retention":
        issues.append("Length Retention Goal")
        simple_score += 1

    # Combined flags
    if scalp == "Dry" and heat == "Often":
        issues.append("HIGH DAMAGE RISK: Dry scalp + frequent heat")
        confidence += 5
        simple_score += 2
    if chemical == "Frequently" and porosity == "High":
        issues.append("SEVERE DAMAGE: Chemical overprocessing + high porosity")
        confidence += 5
        simple_score += 2
    if scalp == "Oily" and chemical == "Frequently":
        issues.append("Scalp imbalance: Oily scalp + chemical treatments")
        confidence += 3
        simple_score += 1
    if damage == "High" and heat == "Often":
        issues.append("CRITICAL: High damage + frequent heat use")
        confidence += 5
        simple_score += 2

    confidence = min(confidence, 95)

    # Hair Profile Scores
    moisture = 80
    if scalp == "Dry": moisture -= 25
    if porosity == "High": moisture -= 20
    if heat == "Often": moisture -= 15
    if porosity == "Low": moisture -= 5
    if environment == "Dry": moisture -= 10
    moisture = max(moisture, 5)

    damage_score = 0
    if heat == "Often": damage_score += 30
    if heat == "Sometimes": damage_score += 15
    if chemical == "Frequently": damage_score += 30
    if chemical == "Occasionally": damage_score += 15
    if porosity == "High": damage_score += 10
    if damage == "High": damage_score += 20
    if damage == "Moderate": damage_score += 10
    damage_score = min(damage_score, 100)

    scalp_health = 85
    if scalp == "Dry": scalp_health -= 30
    if scalp == "Oily": scalp_health -= 20
    if scalp == "Sensitive": scalp_health -= 15
    if chemical == "Frequently": scalp_health -= 15
    scalp_health = max(scalp_health, 5)

    breakage = 0
    if density == "Fine": breakage += 30
    if porosity == "High": breakage += 25
    if chemical == "Frequently": breakage += 25
    if heat == "Often": breakage += 20
    if damage == "High": breakage += 15
    breakage = min(breakage, 100)

    hair_profile = {
        "Moisture Level": moisture,
        "Damage Level": damage_score,
        "Scalp Health": scalp_health,
        "Breakage Risk": breakage
    }

    issues_str = (", ".join(issues) if issues
                  else "No major issues detected")
    return issues, issues_str, confidence, simple_score, hair_profile

# ======================
# HAIR PROFILE DASHBOARD
# ======================
def display_hair_profile(profile):
    st.subheader("Hair Profile Score Dashboard")
    st.caption("Generated by Mini Brain before AI processing")
    col1, col2, col3, col4 = st.columns(4)
    cols = [col1, col2, col3, col4]
    icons = {
        "Moisture Level":  ("Higher = better"),
        "Damage Level":    ("Lower = better"),
        "Scalp Health":    ("Higher = better"),
        "Breakage Risk":   ("Lower = better"),
    }
    for col, (label, value) in zip(cols, profile.items()):
        note = icons[label]
        with col:
            st.metric(label=label, value=f"{value}/100")
            st.progress(value / 100)
            st.caption(note)

# ======================
# STRUCTURED OUTPUT
# ======================
def display_structured_output(recommendation_text):
    sections = [
        ("1. Recommended Shampoo Type",
         "1. Recommended Shampoo Type"),
        ("2. Recommended Conditioner Type",
         "2. Recommended Conditioner Type"),
        ("3. Recommended Treatment",
         "3. Recommended Treatment"),
        ("4. Suggested Weekly Routine",
         "4. Suggested Weekly Routine"),
        ("5. Explanation for Recommendations",
         "5. Explanation for Recommendations"),
    ]
    text = recommendation_text
    extracted = {}
    for i, (key, _) in enumerate(sections):
        if key in text:
```

```python
            after = text.split(key, 1)[1]
            end = len(after)
            for j, (next_key, _) in enumerate(sections):
                if j != i and next_key in after:
                    idx = after.find(next_key)
                    if idx < end:
                        end = idx
            extracted[key] = after[:end].strip()
    if not extracted:
        st.write(recommendation_text)
        return
    for key, display_title in sections:
        if key in extracted:
            with st.expander(display_title, expanded=True):
                st.write(extracted[key])

# ======================
# MAIN BUTTON
# ======================
if st.button("Get Recommendation"):
    detected_issues_simple, confidence_simple = detect_hair_issues(
        scalp_condition, heat_usage,
        chemical_treatments, porosity, density
    )
    issues_list, issues_str, confidence, simple_score, hair_profile = run_mini_brain(
        scalp_condition, heat_usage, chemical_treatments,
        porosity, density, hair_type, hair_goal,
        damage_level, environment
    )

    st.header("Your Personalized Recommendation")
    display_hair_profile(hair_profile)
    st.divider()

    st.subheader("Hair Health Score")
    st.write(f"Score: {simple_score} -- Higher = more attention needed")
    if simple_score >= 8:
        st.error("High attention needed")
    elif simple_score >= 4:
        st.warning("Moderate condition")
    else:
        st.success("Low concern -- maintenance advice")
    st.divider()

    st.subheader("Detected Issues")
    st.write(detected_issues_simple)
    st.divider()

    st.subheader("Mini Brain -- Detailed Issue Breakdown")
    st.caption("Rule-based analysis completed before AI processing")
    if issues_list:
        for issue in issues_list:
            if "RISK" in issue or "SEVERE" in issue or "CRITICAL" in issue:
                st.error(issue)
            else:
                st.warning(issue)
    else:
        st.success("No major issues detected")
    st.divider()

    prompt = f"""
You are a professional licensed trichologist and hair
care expert with 15+ years of experience.

User Profile:
- Hair Type: {hair_type}
- Scalp Condition: {scalp_condition}
- Porosity: {porosity}
- Density: {density}
- Hair Length: {hair_length}
- Damage Level: {damage_level}
- Main Goal: {hair_goal}
- Heat Styling: {heat_usage}
- Chemical Treatments: {chemical_treatments}
- Wash Frequency: {wash_frequency}
- Styling Habits: {styling_habits}
- Environment: {environment}
- Diet & Water Intake: {diet_water}
- Extra Info: {user_description if user_description else "None"}

System Detected Issues: {issues_str}
Hair Health Score: {simple_score}
Confidence Level: {confidence}%

Provide a recommendation using EXACTLY these sections:
1. Recommended Shampoo Type:
2. Recommended Conditioner Type:
3. Recommended Treatment:
4. Suggested Weekly Routine:
5. Explanation for Recommendations:
"""

    client = openai.OpenAI(api_key=api_key)
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    recommendation = response.choices[0].message.content

    st.subheader("AI Recommendation")
    display_structured_output(recommendation)
    st.divider()

    st.progress(confidence / 100)
    st.caption(f"System Confidence Level: {confidence}%")
    st.divider()

    st.subheader("Was this recommendation helpful?")
    rating = st.slider("Rate (1=Not helpful, 5=Excellent)", 1, 5, 4)
    if st.button("Submit Feedback"):
        st.success("Thank you for your feedback!")

# ======================
# SIDEBAR
# ======================
st.sidebar.title("System Architecture")
st.sidebar.write("""
This prototype demonstrates a Decision Support System.

Process:
1 - User answers hair care survey
2 - Streamlit collects inputs
3 - Mini Brain v2.0 detects basic issues
4 - Enhanced Mini Brain calculates weighted scores
5 - Combined condition flags are triggered
6 - Hair Profile Score Dashboard is generated
7 - Hair Health Score is calculated
8 - Python builds a structured AI prompt
9 - OpenAI model analyzes hair characteristics
10 - AI generates personalized recommendations
11 - Results displayed in structured sections
12 - User rates recommendation (feedback loop)
""")

st.sidebar.title("System Features")
st.sidebar.write("""
v4.0 -- All Features Combined:
- Expanded questionnaire (13 questions)
- Hair Profile Score Dashboard
- Hair Health Score (simple number)
- Mini Brain with Confidence Score
- Enhanced Mini Brain (weighted + combined flags)
- Structured Output (5 clear sections)
- Day-by-day Weekly Routine
- Explanation broken down by hair factor
- User Feedback Loop
- Improved Prompt Engineering
""")

st.sidebar.info(
    "This is a hybrid Decision Support System (Rules + AI)"
)
```

Listing 1. Final app.py — AI Hair Care Recommendation System v4.0

Note: The API key shown in the repository version has been replaced with a placeholder. In deployment, the key should be stored securely using environment variables or a secrets management tool, not hardcoded in the source file.

XIV. Evaluation

We tested the system over 13 days and focused on three things:

- **Personalization:** We tried a bunch of different combinations of inputs and checked whether the output actually changed in a meaningful way. Someone with curly, dry hair got completely different suggestions than someone with straight, oily hair — which is exactly what we were going for.
- **Explainability:** Every recommendation came with a reason. For example, the system would say something like "Hyaluronic acid is good for fine curls because it adds moisture without weighing the hair down." That is way more useful than just getting a list of ingredients with no context.
- **Usability:** The app was easy to navigate. The five-section output format made the recommendations feel organized and approachable, and the sidebar explanation helped users understand how the system actually worked.

Accuracy, Response Time, Usability Testing:

Accuracy: The system was tested across 20 different input combinations over the 13-day evaluation period. In all cases, the AI produced recommendations that were logically consistent with the inputs provided — users with dry, curly hair consistently received moisture-focused routines while users with oily scalps received clarifying-focused advice. Because there is no labeled ground truth dataset for personalized hair care recommendations, we evaluated accuracy qualitatively by checking whether the output matched what a knowledgeable hair care advisor would reasonably suggest for each profile. All 20 test cases produced relevant, on-topic recommendations with no hallucinated or contradictory advice detected.

Response Time: API response times were measured across 15 test runs. The average time from survey submission to full recommendation display was approximately 4.2 seconds, with a minimum of 2.8 seconds and a maximum of 6.1 seconds. Response time varied slightly depending on network conditions and the length of the user's optional text description. All responses were considered acceptable for a conversational AI tool.

Usability Testing: The app was informally tested by 5 users with varying levels of tech experience. All 5 were able to complete the survey and receive a recommendation without any assistance. Common feedback included that the five-section output format was easy to follow and that the explanation section was the most useful part. One user noted they wished they could save their results, which has been identified as a future improvement.

Comparative Analysis: Our AI Hair Assistant provides a service to users that is typically provided in-person at a hair salon or by a hair specialist, so it is difficult to present a direct comparative analysis. What can be noted is that the AI Hair Assistant does make personalized hair recommendations more accessible for end users who may not have a salon within a reasonable distance of them or those who may not be able to afford a salon's service price.

In contrast to the accessibility, the ability to visit a hair stylist in-person allows for a live demonstration of hair styling techniques that the AI Hair Assistant can only describe. The user may take further steps to find an online video or more in depth description of the technique, but a hair stylist could simply explain the procedure while actively performing it. Another factor to consider in favor of using an in-person hair stylist is that the end user may incorrectly self-report issues about their hair symptoms. There is room in the hair questionnaire for the user to select the incorrect hair type, texture, porosity, and so on which would then cause an irrelevant response. A licensed hair stylist should have a smaller margin of error when recognizing issues and/or qualities about a client's hair.

Some things we know the system still cannot do: it does not recommend specific brands, it does not remember anything between sessions, and it depends on the OpenAI API being available. Those are all things we would want to improve in a future version.

XV. Conclusion and Future Work

We built a working AI system that gives people personalized hair care advice based on a short survey. The combination of the Mini Brain module and GPT-4o-mini gave us something that is both reliable and flexible — it handles simple cases quickly and can still reason through complicated hair situations that a rule-based system would struggle with.

Phase 1 was about identifying the problem, doing the research, and getting a first version of the app working. Phase 2 was about formalizing the design with UML diagrams, improving the prototype, and actually evaluating whether it worked.

There is definitely more we want to do with this. Some ideas for where to take it next:

1) Connect it to a real product database so it can recommend specific brands, not just product types
2) Add the ability to upload a photo of your hair so the AI can visually analyze it
3) Save user sessions so the system can track progress and adjust recommendations over time
4) Run a formal user study to get real feedback from people with different hair types

References

[1] D. Roy and M. Dutta, "A systematic review and research perspective on recommender systems," *Journal of Big Data*, vol. 9, no. 59, 2022. [Online]. Available: https://link.springer.com/article/10.1186/s40537-022-00592-5

[2] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Computing Surveys*, vol. 52, no. 1, pp. 1–38, Feb. 2019. [Online]. Available: https://arxiv.org/pdf/1707.07435.pdf

[3] A. Alharbi *et al.*, "Optimized personalized therapy in clinical decision support system using NLP," *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 3, Mar. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1319157820303268

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019. [Online]. Available: https://arxiv.org/pdf/1810.04805

[5] A. Holzinger, C. Biemann, C. S. Pattichis, and D. B. Kell, "What do we need to build explainable AI systems for the medical domain?" *arXiv preprint arXiv:1806.00069*, 2019. [Online]. Available: https://arxiv.org/pdf/1806.00069